

EpiGen: where the wall-clock time actually goes

August 15, 2026

Why this document exists

A run was timed end to end on the whole-loop-on-Modal path (`use_modal_mcmc=True`) to answer “what’s taking so long.” This is that measurement, what it does and doesn’t tell us, and the concrete levers available to cut it down.

Contents

1	The measured run	1
2	The two-bucket breakdown	1
3	Why the inner 872s is plausibly real compute, not something broken	2
4	Levers to actually cut wall time	2
5	What would give a real per-expert breakdown	3

1 The measured run

Config	Value
Scaffold	193L (lysozyme, 129 aa)
Edit	position 60, WHSPRAL (7 residues)
Window	full protein minus the edit (122 positions)
<code>num_starting_points × chains_per_start</code>	$20 \times 2 = 40$ chains
<code>steps</code>	20 (upper bound; chains freeze early)
<code>candidate_num</code>	5
<code>use_modal_mcmc</code>	<code>True</code> (whole search loop runs inside one Modal call)

Total wall time: 1075.7s (~18.0 min). Result: 5 candidates, `wt_score=720.210`, `wt_nt_source=codonfm` (193L has no verified GenBank CDS for this sequence – Evo2’s nt sequence came from the CodonFM fallback, not a real deposited coding sequence; see `oracle.nt_resolution`).

2 The two-bucket breakdown

The client-visible log only prints a timer around each *separate* Modal dispatch (“Connecting to modal container: 0it [XXs, ?it/s]”). Summing the non-zero ones gives the overhead *outside* the search loop itself; the remainder is time spent inside the one, single Modal call that runs the entire MCMC search server-side.

Bucket	Time	Share
Visible container-connect overhead (sum of non-zero “Connecting...” timers)	~204s	19%
Inside the single <code>run_mcmc_search_remote</code> call (no separate connect events)	~872s	81%
Total	1075.7s	100%

The 204s bucket is the one-off calls *around* the search: the initial fold, ProteinMPNN warm-start generation (`propose_compensatory_mutations`), the pre/post-search ESM2+ProteinMPNN oracle-sanity-check batch calls, and refold+TM-align on the top candidates. This is exactly the overhead `use_modal_mcmc=True` was built to shrink for the *search itself* (collapsing per-round laptop↔Modal round trips into one container-to-container loop) – and it did: none of the 20 MCMC rounds shows up here as a separate connect event, because they’re not separate dispatches anymore.

The 872s bucket – the large majority of the wall time – is therefore *not* network/dispatch overhead. It’s real compute time inside one container: up to 20 rounds, each round scoring up to 40 chains’ proposals with three experts (ESM2, ProteinMPNN, Evo2).

Caveat: This split is derived from the client-side log’s dispatch timers, not from any instrumentation inside `run_mcmc_search/ run_mcmc_search_remote` itself. It tells us *overhead vs. everything else*, not which of the three experts (or which round) dominates the 872s. See §5 for what real per-expert numbers would take.

3 Why the inner 872s is plausibly real compute, not something broken

Three things compound inside that one call:

- **Round count is worst-case, not average-case.** The round loop only exits early once *every* chain has frozen (its own score exceeded `wt_score`). A handful of slow-to-converge chains out of 40 can force close to the full 20 rounds even if most chains froze much earlier.
- **Every active round pays three full batched model calls** – `position_scores_esm2_batch`, `position_scores_proteinmpnn_batch`, and `window_log_prob_batch` (Evo2) – each over every still-active chain’s proposal.
- **Evo2’s call is the heaviest of the three per-token.** As of today’s window-restriction fix, it runs with `return_logits=True` – a full (`seq_len`, `vocab_size`) logits tensor per sequence, not just a scalar `avg_log_likelihood` – so it’s doing meaningfully more work per call than the old whole-sequence-average version, in exchange for being scoped correctly (window-only, matching ESM2/ProteinMPNN) instead of diluted over the whole sequence.

None of this is a bug; it’s the cost of the fix instead of a symptom of it – the network overhead is gone, what’s left is the model compute the search was always going to need for a 40-chain,

up-to-20-round, 3-expert search.

4 Levers to actually cut wall time

Lever	Effect
Lower <code>steps</code>	Directly caps worst-case rounds; the freeze mechanism exits early for chains that converge, so this mostly trims the tail.
Fewer chains (<code>num_starting_points</code> $\times$ <code>chains_per_start</code>)	Shrinks every round's batch size linearly; 40 chains is the end of what's been run this session.
Raise <code>weight_evo2</code> or tune <code>temperature</code>	Changes how fast chains cross the freeze threshold – the effective round count without touching <code>steps</code> changes what's being optimized.
Drop Evo2 (<code>use_evo2=False</code>)	Removes the heaviest per-round call entirely – back to search, faster but loses the DNA-level signal.

5 What would give a real per-expert breakdown

Right now there is no timestamp recorded around the individual ESM2 / ProteinMPNN / Evo2 calls *inside* `run_mcmc_search`, on either the laptop-orchestrated or whole-loop-on-Modal path. Getting genuine per-expert numbers (not just “inside vs. outside the big call”) would mean adding `time.time()` timers around each of the three batched calls in `oracle/mcmc.py`'s round loop, aggregating them (e.g. into the same checkpoint progress payload `oracle/checkpoint.py` already writes), and re-running. Not done here – this document is the coarse split available from the existing log output, not a profiling pass.